Соглашение по C++ для учебного курса

Элемент	Формат	Пример
Класс, структура, объединение	PascalCase	`StudentReport`, `GradeRecord`
Перечисление	PascalCase; предпочтительно enum class	`ReportStatus`
Элемент перечисления	PascalCase	`ReportStatus::Ready`
Псевдоним типа	PascalCase	`GradeList`, `CountType`
Параметр шаблона, обозначающий тип	PascalCase	`ValueType`
Свободная функция	PascalCase	`CalculateAverage`
Любой метод класса	PascalCase	`Print`, `ValidateGrades`
Геттер, сеттер, проверка условия	PascalCase; Get / Set / Is / Has	`GetName`, `SetName`, `IsEmpty`
Локальная переменная	camelCase	`totalGrade`
Параметр функции	camelCase	`studentCount`
Нестатическое поле класса	m_ + camelCase	`m_studentCount`
Открытое поле простой структуры	camelCase	`gradeCount`
Изменяемое статическое поле класса	s_ + camelCase	`s_createdCount`
Именованная константа	c_ + camelCase	`c_maxGrade`
Пространство имен	camelCase	`courseStyle`
Макрос, если он необходим	UPPER_SNAKE_CASE	`COURSE_PLATFORM_WINDOWS`
Файл и каталог исходного кода	snake_case; .h / .cpp	`student_report.h`, `src`

1. Как читать и применять это соглашение

Основа именования — соглашения Microsoft DirectXTK. Скобки и отступы — профиль Microsoft в clang-format с настройками курса. Microsoft выпускает разные библиотеки с разными соглашениями; этот документ фиксирует один конкретный вариант для всех работ курса. Порядок членов класса, обязательность префиксов, имена файлов и запрет комментариев — решения курса. Стандарт языка выбран C++17; другой стандарт применяется только по условию задания.

PascalCase начинает каждое слово с большой буквы: `StudentReport`. camelCase начинает первое слово с маленькой буквы: `studentReport`. Подчеркивания используются в установленных префиксах `m_`, `s_`, `c_`, в макросах и именах файлов.

Имена пишутся на английском языке и обозначают смысл. Все собственные методы имеют один формат: ни доступ public/private, ни наличие virtual/override его не меняют. Имена стандартной библиотеки и внешнего API сохраняются: `std::vector`, `size`, `push_back`. Особые функции `main` и операторы сохраняют требуемую языком форму. При переопределении метода внешнего класса его имя и сигнатуру менять нельзя.

В собственных исходниках курса не используются комментарии: ни `//`, ни `/* ... */`, ни документационные комментарии, ни TODO, ни закомментированный код. Объяснение решения, алгоритма и ограничений помещается в отчет или README. Хорошие имена передают назначение, но математическое обоснование и внешние требования при необходимости описываются отдельно. Лицензионные уведомления чужого кода сохраняются; сторонние библиотеки не переписываются под курс.

Каждый самостоятельный класс, структура, объединение и перечисление объявляется в отдельном заголовке. Размер типа не является исключением: даже небольшое перечисление получает свой файл. Имя файла приводится над блоком кода, вне исходника; два файла никогда не объединяются в один блок. Короткие примеры отдельных выражений и методов — фрагменты соответствующего файла, а не его полное содержимое.

2. Пояснения к каждой строке таблицы

2.1. Класс, структура, объединение

Тип называется существительным или сочетанием существительных в единственном числе. `class` используется для объекта с поведением и ограничениями состояния; `struct` — для простого набора открытых данных. `union` относится к тому же правилу именования, но в обычных заданиях вместо него предпочтителен `std::variant`, если требуется хранить разные варианты значения.

Файл: `include/grade_record.h`

```
#pragma once

struct GradeRecord
{
    int grade = 0;
};
```

2.2. Перечисление

Название перечисления описывает категорию значений. Используется `enum class`: его значения принадлежат области имени типа и не превращаются неявно в целые числа. Обычный `enum` допускается для совместимости с заданным API.

Файл: `include/report_status.h`

```
#pragma once

enum class ReportStatus
{
    Ready,
    Empty,
    Failed
};
```

2.3. Элемент перечисления

Значение пишется в PascalCase. В названии не нужно повторять имя перечисления: область `ReportStatus::` уже дает контекст. Перечислитель — именованное значение своего типа; он не получает префикс `c_`.

```
ReportStatus reportStatus = ReportStatus::Ready;
```

2.4. Псевдоним типа

Псевдоним обозначает тип, поэтому имеет PascalCase. Для нового кода применяется `using`, а не `typedef`. Псевдоним полезен, если выражает роль типа или упрощает сложную запись, а не скрывает простой тип без причины.

```
using GradeList = std::vector<int>;
using CountType = std::size_t;
```

2.5. Параметр шаблона, обозначающий тип

Имя описывает роль типа. `ValueType`, `KeyType` и `Predicate` понятнее произвольных букв в учебном прикладном коде. Параметр шаблона, обозначающий значение, получает camelCase: правило для типа на него не переносится.

```
template <typename ValueType, std::size_t elementCount>
using FixedArray = std::array<ValueType, elementCount>;
```

2.6. Свободная функция

Функция называется действием. Возвращаемый результат должен быть понятен из имени: `CalculateAverage` вычисляет среднее, `LoadGrades` загружает оценки. `DoWork`, `ProcessData` и `Func1` не объясняют конкретную задачу.

```
double CalculateAverage(const std::vector<int>& grades);
```

2.7. Любой метод класса

Для public, protected и private действует PascalCase. То же относится к static, virtual и override. Метод является функцией класса; отдельный формат для приватных методов не вводится.

```
public:
    void Print(std::ostream& stream) const;

private:
    static void ValidateGrades(const GradeList& grades);
```

2.8. Геттер, сеттер, проверка условия

Геттер начинается с `Get`, сеттер с `Set`; логический вопрос — с `Is`, `Has` или `Can`. Геттер не должен неожиданно изменять наблюдаемое состояние. Сеттер проверяет ограничения. Не нужно автоматически создавать сеттер для каждого поля: метод `AddGrade` может лучше сохранять корректность объекта, чем выдача изменяемого списка наружу.

```
[[nodiscard]] int GetStudentCount() const noexcept;
void SetStudentCount(int studentCount);
[[nodiscard]] bool IsEmpty() const noexcept;
[[nodiscard]] bool HasGrades() const noexcept;
```

2.9. Локальная переменная

Имя объясняет хранимое значение и, когда это важно, единицы измерения. `elapsedMilliseconds`, `studentCount` и `averageGrade` предпочтительнее `time`, `n` и `result`. Однобуквенный индекс допустим в коротком очевидном цикле; индекс предметного объекта лучше назвать `studentIndex`. Префиксы типов вроде `iCount`, `strName` не используются. Сокращения оформляются как слова: `studentId`, `HttpClient`, `GetUrl`.

```
double totalGrade = 0.0;
int elapsedMilliseconds = 250;
bool hasGrades = !grades.empty();
```

2.10. Параметр функции

Параметр имеет camelCase и обозначает то, что получает функция. Имена в объявлении и определении совпадают. Префикс поля позволяет использовать естественное имя параметра без суффиксов `New`, `Value` и `Param`.

```
void SetName(std::string name)
{
    m_name = std::move(name);
}
```

2.11. Нестатическое поле класса

Префикс `m_` показывает, что значение принадлежит конкретному объекту. Поля класса размещаются в private; наследник работает через методы, а не напрямую меняет состояние базы. Если по условию действительно требуется protected-поле, оно также получает `m_`.

```
private:
    int m_studentCount = 0;
    std::string m_name;
```

2.12. Открытое поле простой структуры

Для структуры, которая служит набором данных без скрытого состояния, используются camelCase и открытые поля. Префикс `m_` здесь не нужен. Если появляются сложные проверки и зависимые поля, стоит выделить класс с методами, сохраняющими его ограничения.

Файл: `include/student_summary.h`

```
#pragma once

struct StudentSummary
{
    int studentCount = 0;
    double averageGrade = 0.0;
};
```

2.13. Изменяемое статическое поле класса

Статическое поле общее для всех объектов, поэтому получает `s_`. Оно размещается в private. Изменяемое общее состояние создается только при обоснованной необходимости; обычный счетчик отдельного объекта остается `m_`. В C++17 `inline static` позволяет определить поле непосредственно в классе.

```
private:
    inline static int s_createdCount = 0;
```

2.14. Именованная константа

Для именованных констант предметной области применяется `c_`: границы оценок, фиксированные размеры, коэффициенты. Для значения, вычислимого при компиляции, используется `constexpr`. Константы класса обычно объявляются `static constexpr`. `const` запрещает изменение, но сам по себе не означает вычисление при компиляции.

```
constexpr int c_minGrade = 2;
constexpr int c_maxGrade = 5;
const int studentCount = ReadStudentCount();
```

Последняя строка — обычная локальная переменная, значение которой получили во время работы программы и далее не меняют: она остается `studentCount`. `constexpr`-переменные,

статические локальные константы и константы уровня пространства имен имеют `c_`. Константа состояния отдельного объекта остается полем `m_`; константа, общая для типа, имеет `c_`. Не нужно объявлять константу ради каждого `0`, `1` или текста сообщения: имя полезно, когда объясняет смысл значения.

2.15. Пространство имен

Для собственного пространства имен применяется camelCase. Его имя обозначает модуль или предметную область. `using namespace std;` не используется. Короткие локальные псевдонимы допустимы, если улучшают читаемость и не загрязняют заголовки.

```
namespace courseStyle
{
double CalculateAverage(const std::vector<int>& grades);
}
```

2.16. Макрос

Макрос получает UPPER_SNAKE_CASE и префикс проекта, если может конфликтовать с внешним кодом. Для констант, функций и типов применяются средства языка; макрос остается для условной компиляции и совместимости. Макросы не подменяют ключевые слова и операции.

```
#if defined(COURSE_PLATFORM_WINDOWS)
#include <Windows.h>
#endif
```

2.17. Файл и каталог

Название файла отражает основную сущность: класс `StudentReport` хранится в `student_report.h` и `student_report.cpp`. Заголовки помещаются в `include`, реализации — в `src`. Имена без пробелов, кириллицы и различий только в регистре удобны на всех платформах. Это правило курса, а не общее требование Microsoft.

```
include/student_report.h
src/student_report.cpp
src/main.cpp
```

3. Правильное объявление класса: сначала образец

Ниже два связанных класса из готового проекта. У каждого собственный заголовок и отдельный блок кода. Несмотря на связь классов, объединять их объявления в одном файле нельзя. Сначала показан файл базового интерфейса, затем файл конкретной реализации. Оба примера содержат полные заголовки, включая подключения.

Файл: `include/report.h`

```
#pragma once

#include <cstddef>
#include <iosfwd>

namespace courseStyle
{
class Report
{
public:
    Report(const Report&) = delete;
    Report(Report&&) = delete;

    virtual ~Report() = default;

    Report& operator=(const Report&) = delete;
    Report& operator=(Report&&) = delete;

    void Print(std::ostream& stream) const;

    [[nodiscard]] virtual std::size_t GetRowCount() const noexcept = 0;

protected:
    Report() = default;

    virtual void PrintBody(std::ostream& stream) const = 0;
};
}
```

Файл: `include/student_report.h`

```
#pragma once

#include "report.h"

#include <cstddef>
#include <iosfwd>
#include <vector>

namespace courseStyle
{
class StudentReport final : public Report
{
public:
    using GradeList = std::vector<int>;

    static constexpr int c_minGrade = 2;
    static constexpr int c_maxGrade = 5;

    explicit StudentReport(GradeList grades);

    ~StudentReport() override = default;

    [[nodiscard]] const GradeList& GetGrades() const noexcept;
    [[nodiscard]] double GetAverage() const;

    [[nodiscard]] std::size_t GetRowCount() const noexcept override;

protected:
    void PrintBody(std::ostream& stream) const override;

private:
    static void ValidateGrades(const GradeList& grades);

    GradeList m_grades;
};
}
```

3.1. Строка class, имя, final и наследование

`class StudentReport` вводит тип. `final` запрещает дальнейшее наследование от него: здесь тип является конечной реализацией отчета. Он не обязателен для любого класса; добавляется, когда расширение наследованием не предусмотрено. `: public Report` означает, что StudentReport можно использовать через интерфейс Report. Private-наследование решает другую задачу и не подставляется вместо public только ради сокрытия полей.

3.2. Секции public, protected, private

Секции идут в порядке public → protected → private. Public описывает интерфейс пользователя класса. Protected предоставляет точки расширения наследникам. Private хранит детали реализации. Пустые секции не пишутся; каждая непустая секция объявляется один раз. В class доступ по умолчанию private, в struct — public, но для класса секции указываются явно.

3.3. Порядок внутри каждой секции

В каждой секции применяется один и тот же порядок. Если категория отсутствует, место для нее не оставляют:

1. Псевдонимы и вложенные типы.
2. Именованные константы.
3. Конструкторы: обычные, затем копирующий и перемещающий, если они объявляются явно.
4. Деструктор.
5. Копирующее и перемещающее присваивание.
6. Обычные методы: сначала чтение состояния, затем действия и изменение состояния.
7. Вновь объявляемые виртуальные методы, включая чисто виртуальные.
8. Переопределения унаследованных методов.
9. Остальные операторы, если они нужны.
10. Изменяемые статические поля.
11. Нестатические поля.

Деструктор с override остается среди деструкторов, а не переезжает к остальным переопределениям. Static-метод относится к обычным методам. Между категориями оставляется одна пустая строка. Внутри категории связанные операции идут вместе; сортировка по алфавиту не требуется. Это соглашение о чтении класса, а не обязательный порядок языка.

3.4. Псевдоним using GradeList

Псевдоним стоит первым, поскольку последующие объявления его используют. Он находится в public: вызывающий код может назвать тип `StudentReport::GradeList`. Внутренний псевдоним, не нужный пользователю, помещается в private, также в начале этой секции.

3.5. Константы c_minGrade и c_maxGrade

Они идут после типов и перед конструкторами. `static` делает значение общим для класса, `constexpr` позволяет использовать его как константное выражение. Эти границы можно открыть пользователю, чтобы он заранее проверил данные. Константа внутреннего алгоритма размещается в private. Открытые константы не равнозначны открытым изменяемым полям.

3.6. Конструктор, explicit и список инициализации

Конструктор задает корректное начальное состояние. Конструктор с одним обязательным аргументом объявляется explicit, если неявное преобразование специально не предусмотрено. Поля инициализируются в списке инициализации, а не сначала создаются и затем присваиваются в теле. Список повторяет порядок объявления полей: именно порядок объявления определяет очередность инициализации.

```
StudentReport::StudentReport(GradeList grades) : m_grades(std::move(grades))
{
    ValidateGrades(m_grades);
}
```

Аргумент получен по значению, поскольку объект сохраняет его у себя. `std::move` позволяет передать буфер в поле. После переноса нельзя полагаться на прежнее содержимое `grades`; проверяется уже `m_grades`. Ошибка проверки не позволяет создать некорректный объект.

3.7. Защищенный конструктор базового класса

`Report() = default` находится в protected: базовую часть создают наследники, а пользователю она как отдельный объект не предназначена. Report также абстрактен из-за чисто виртуальных методов. Доступ конструктора и абстрактность — разные свойства. Конструкторы не бывают virtual.

3.8. Копирование, перемещение и operator=

В базе копирование и перемещение явно запрещены с помощью `= delete`. Запрет не допускает случайного копирования через базовый интерфейс и относится к дизайну этого полиморфного типа. `operator=` — специальное имя языка; PascalCase к нему не применяется. Конструкторы копирования и перемещения стоят среди конструкторов, операторы присваивания — после деструктора.

Обычный класс с `std::vector`, `std::string` и другими самостоятельно управляемыми полями чаще не объявляет ни деструктор, ни копирование, ни перемещение: применяется правило нуля. Если специальная операция требуется вручную, нужно рассмотреть все пять операций: деструктор, копирующий и перемещающий конструкторы, оба присваивания. Это не требование писать пять функций в каждом классе. Пользовательское объявление деструктора влияет на автоматическое создание операций перемещения.

3.9. Деструктор базы и override у наследника

`virtual ~Report() = default` позволяет корректно уничтожить наследника через указатель на Report. Он открыт, потому что пользователю разрешено такое уничтожение. Деструктор наследника `~StudentReport() override = default` подтверждает переопределение виртуального деструктора. Ненужное пустое тело заменяется на `= default`.

```
std::unique_ptr<Report> report =
    std::make_unique<StudentReport>(StudentReport::GradeList{4, 5, 3});
```

Для базового класса выбирается подходящая модель уничтожения: открытый виртуальный деструктор либо защищенный невиртуальный, если уничтожение через базу намеренно запрещено. В рабочем примере нужна первая модель. Виртуальные вызовы из конструктора и деструктора не используются для обращения к поведению полностью построенного наследника.

3.10. Обычные публичные методы

`GetGrades` и `GetAverage` дают чтение состояния. `const` после скобок означает, что метод не изменяет обычные поля объекта и может быть вызван у const-объекта. `GetGrades` возвращает const-ссылку, чтобы не копировать список и не открывать его изменение. Такая ссылка действительна только пока существует объект и сохраняется соответствующее хранилище; ссылку на данные временного объекта сохранять нельзя.

`Print` в базе — обычный метод. Он проверяет наличие строк и вызывает виртуальный `PrintBody`. Общая последовательность действий задается базой, конкретное содержимое выводит наследник.

Геттер не всегда обязан быть дешевым: здесь `GetAverage` вычисляет среднее; стоимость операции должна быть понятна из контекста, а при сложном вычислении лучше имя `CalculateAverage`.

3.11. Новый virtual и чисто виртуальный метод = 0

`virtual` пишется при введении виртуального метода в базе. Суффикс `= 0` означает, что метод чисто виртуальный; класс с не реализованными чисто виртуальными методами абстрактен, объект такого класса создать нельзя. Наследник становится конкретным, когда реализованы все необходимые методы.

```
virtual void PrintBody(std::ostream& stream) const = 0;
```

Public-метод `GetRowCount` доступен пользователю интерфейса. Protected-метод `PrintBody` предназначен для наследников. Чисто виртуальный метод также может быть private: его разрешено переопределить, хотя вызвать через private-интерфейс базы снаружи нельзя. Поэтому абстрактность не определяет секцию доступа. В этом курсе точки расширения обычно protected; private virtual применяется только при осознанной конструкции интерфейса. Чисто виртуальному деструктору, если он введен, все равно требуется определение; в данном примере это усложнение не нужно.

3.12. Переопределение override и final у метода

В наследнике используется override, без повторного virtual. Компилятор проверит совпадение сигнатуры с виртуальным методом базы: типы параметров, const и другие значимые свойства. Одна только похожесть имени не гарантирует переопределения.

```
std::size_t GetRowCount() const noexcept override;
```

`final` у метода запрещает его дальнейшее переопределение. Если требуется и проверка переопределения, и запрет, пишется `override final`. В final-классе дополнительно ставить final на каждом методе не нужно. Доступ override в наследнике и доступ виртуального метода в базе проверяются по тому интерфейсу, через который написан вызов; private override не делает public-интерфейс базы закрытым.

3.13. const, noexcept и nodiscard

В объявлении порядок такой: атрибут, тип результата, имя и параметры, const, noexcept, override, final. `[[nodiscard]]` указывает, что игнорирование результата подозрительно; он применяется к важным результатам, а не ко всем функциям автоматически. `noexcept` — обещание не выпускать исключение: если оно нарушено, программа завершится через `std::terminate`.

```
[[nodiscard]] std::size_t GetRowCount() const noexcept override;
```

`GetRowCount` можно объявить noexcept. `GetAverage` бросает исключение для пустого списка, поэтому noexcept у него нет. При переопределении метода с noexcept нельзя ослабить это обещание. В определении вне класса const и noexcept повторяются; virtual, override и final не повторяются.

3.14. Приватные вспомогательные методы

`ValidateGrades` расположен в private: это деталь создания корректного отчета. Он static, потому что использует аргумент и константы класса, но не состояние конкретного объекта. Static-метод не имеет `this` и не может быть virtual. Название остается PascalCase.

3.15. Поля в конце секции и определение методов

`m_grades` находится в конце private. Порядок полей отражает их зависимость при инициализации; перестановка в списке конструктора его не меняет. Пустые стандартные контейнеры и строки уже имеют корректное начальное состояние. Числа, указатели и перечисления получают явные начальные значения, если конструктор не задает их обязательно.

Короткое тривиальное определение допустимо в классе; значимая логика переносится в .cpp. Определения идут в порядке объявлений, чтобы файл было проще просматривать. Шаблоны обычно определяются в заголовках, где их реализация доступна при создании специализаций.

4. Оформление исходного кода

4.1. Отступы, скобки и длина строки

Отступ — четыре пробела, табуляция не используется. Открывающая фигурная скобка стоит на следующей строке: стиль Allman. Тело класса и функции получает один дополнительный отступ. Метки доступа стоят на уровне class. Внутри namespace дополнительного отступа нет. Целевая длина строки — 100 символов; clang-format переносит конструкцию по своим правилам, а длинный неделимый текст или имя может превысить предел.

```
if (studentCount > 0)
{
    PrintReport();
}
else
{
    PrintEmptyMessage();
}
```

4.2. Фигурные скобки у условий и циклов

Скобки обязательны даже для одного действия. Это делает границы блока видимыми и защищает от ошибки при добавлении следующей строки. Тело функции также не сворачивается в одну строку. Пустой блок не заменяет необъясненное отсутствие обработки ошибки.

```
for (int grade : grades)
{
    totalGrade += grade;
}
```

4.3. Пробелы и пустые строки

После запятой и вокруг бинарных операторов ставится пробел, между именем функции и `(` — нет. Между `if`, `for`, `while`, `switch` и `(` — пробел. Унарные `!`, `++` не отделяются от операнда. Несвязанные этапы разделяются одной пустой строкой; несколько пустых строк подряд, пробелы в конце строк и ручное выравнивание десятками пробелов не нужны.

```
double averageGrade = totalGrade / studentCount;
if (!hasGrades)
{
    return;
}
```

4.4. Указатели, ссылки и переносы

Знак `*` или `&` примыкает к типу. На одной строке объявляется одна переменная: это исключает двусмысленное чтение `int* first, second`. Положение переносов параметров, списков инициализации и длинных выражений задает clang-format; ручное форматирование не должно с ним спорить.

```
const StudentReport& reportView = report;
const StudentReport* reportPointer = nullptr;
```

4.5. Файлы, кодировка и завершение строки

Файлы сохраняются в UTF-8, завершаются переводом строки. В одном репозитории используется единая политика перевода строк; для нового кроссплатформенного проекта выбирается LF. Не смешиваются разные переводы строк внутри файла. Собственные исходники не содержат комментариев, включая комментарий после закрывающей скобки namespace. Настройка `FixNamespaceComments: false` предотвращает их автоматическое добавление, но не удаляет существующие комментарии.

5. Заголовки и устройство проекта

5.1. Сначала собственный заголовок реализации

Файл .cpp первым подключает соответствующий .h: так обнаруживается, если заголовок сам не подключил необходимую зависимость. Далее идут остальные заголовки проекта в кавычках, затем библиотечные заголовки в угловых скобках; группы разделяются пустой строкой. Внутри группы применяется сортировка clang-format. Особый порядок сохраняется только там, где он требуется API.

```
#include "student_report.h"

#include <ostream>
#include <stdexcept>
#include <utility>
```

5.2. Заголовок самодостаточен

Собственный заголовок имеет `#pragma once` и подключает все, что необходимо для его объявлений. Не используется случайное транзитивное подключение: применение `std::vector` требует `<vector>`. Для ссылки на поток без его использования может быть достаточно `<iosfwd>`; определения методов вывода подключают `<ostream>`. В рамках курса используется pragma once, хотя формально это широко поддерживаемое расширение компиляторов. Если требуется строгое переносимое окружение, применяют include guard с уникальным макросом.

```
#pragma once

#include <cstddef>
#include <vector>
```

5.3. Объявление и определение

Объявление сообщает компилятору, что существует функция с определенным именем, параметрами и возвращаемым типом. Определение содержит ее тело: действия, которые выполняются при вызове. Обычную свободную функцию объявляют в .h, а определяют в одном .cpp. Сначала рассмотрим правильное разделение по файлам.

Файл: `include/grade_rules.h`

```
#pragma once

bool IsPassingGrade(int grade, int minimumGrade = 3);
```

Файл: `src/grade_rules.cpp`

```
#include "grade_rules.h"

bool IsPassingGrade(int grade, int minimumGrade)
{
    return grade >= minimumGrade;
}
```

Файл: `src/main.cpp`

```
#include "grade_rules.h"

int main()
{
    return IsPassingGrade(4) ? 0 : 1;
}
```

Что находится в .h. Строка с IsPassingGrade заканчивается точкой с запятой и не содержит тела. Это объявление: любой .cpp, подключивший заголовок, узнает, как вызвать функцию. В заголовках также размещают определения классов, структур и перечислений: компилятору нужно знать их устройство. Поэтому правило «в .h только объявления» слишком грубое. Заголовок обычно содержит определение класса, а внутри него — объявления его методов.

Что находится в .cpp. Фигурные скобки с return образуют тело функции. Это определение. Оно должно соответствовать объявлению по типам параметров и возвращаемому типу. Собственный заголовок подключается первым, чтобы компилятор проверил это соответствие. Имена параметров в объявлении и определении технически могут отличаться, но в курсе сохраняются одинаковыми.

Как файлы объединяются. Каждый .cpp компилируется отдельно. После этого компоновщик соединяет результаты и связывает вызов IsPassingGrade из main.cpp с определением из grade_rules.cpp. В сборку должны входить оба .cpp. Если включить только main.cpp, объявление будет найдено, но определения функции для компоновки не хватит.

Почему нельзя подключать .cpp. Директива include фактически вставляет содержимое подключаемого файла. Поэтому подключают .h, а .cpp передают системе сборки как отдельные исходные файлы. Подключение .cpp может привести к тому, что одно определение попадет в несколько результатов компиляции.

Почему тело обычной функции не помещают в .h. Если заголовок с таким телом подключат два .cpp, в программе появятся два определения одной функции. Обычно компоновщик сообщит об ошибке множественного определения. `#pragma once` предотвращает повторное подключение внутри одного компилируемого .cpp, но не устраняет эту проблему между разными .cpp.

Как разделять методы класса. Класс объявляет доступные операции в своем .h, а определения обычных методов размещаются в соответствующем .cpp. В определении имя класса и `::` показывают, какому классу принадлежит метод.

Файл: `include/student.h`

```
#pragma once

class Student
{
public:
    explicit Student(int age);

    int GetAge() const;

private:
    int m_age;
};
```

Файл: `src/student.cpp`

```
#include "student.h"

Student::Student(int age)
    : m_age(age)
{
}

int Student::GetAge() const
{
    return m_age;
}
```

У GetAge суффикс const присутствует и в объявлении, и в определении. Аналогично сохраняется спецификация noexcept, если она есть. Ключевые слова virtual, override и static для метода указываются в объявлении внутри класса; повторять их в определении вне класса не нужно.

Когда определение допустимо в заголовке. В рамках курса это определения шаблонов, короткие методы внутри определения класса, функции constexpr и специально объявленные inline-функции. Метод, определенный внутри класса в обычном заголовке, неявно является inline. Функция constexpr также неявно является inline. Для таких функций допускается одинаковое определение в нескольких единицах трансляции при соблюдении правил C++. Здесь inline решает вопрос допустимости определений, а не гарантирует ускорение или подстановку тела вместо вызова. Обычные нешаблонные функции по умолчанию остаются в .cpp; не следует добавлять inline только ради переноса всех реализаций в заголовки.

Почему шаблоны обычно остаются в .h. Чтобы создать нужный вариант шаблона, например для int или double, компилятору обычно требуется видеть его определение в месте использования. Разделение шаблонов с явным инстанцированием возможно, но не является стандартной схемой учебных работ.

Где задавать значения по умолчанию. В примере minimumGrade = 3 указано один раз в объявлении в .h. В определении в .cpp остается просто int minimumGrade. Вызов IsPassingGrade(4) использует значение 3, а IsPassingGrade(4, 5) передает свое значение. Для виртуальных методов в курсе аргументы по умолчанию не задаются: они выбираются по статическому типу выражения, тогда как реализация метода — по фактическому типу объекта, что может запутать вызывающий код.

Что делать с переменными. Изменяемые глобальные переменные в заголовках курса запрещены: состояние хранится в объектах и передается через параметры. Для именованной константы в заголовке допустимо inline constexpr; это отличается от изменяемой глобальной переменной.

Имена файлов отражают назначение и не содержат номер версии или суффикс final_final. Каждый самостоятельный тип размещается отдельно по правилу 5.4; grade_rules.h в примере содержит свободную функцию, поэтому это правило не требует создания класса GradeRules.

5.4. Один файл — один самостоятельный тип

Каждый самостоятельный класс, структура, объединение и перечисление получает свой .h-файл. Это обязательное правило курса, включая маленькие вспомогательные структуры и перечисления. Связь типов между собой не разрешает объединять их в одном заголовке. Вложенный тип остается внутри своего внешнего типа: он является частью его объявления.

Класс с определениями методов вне заголовка получает соответствующий .cpp-файл. Перечислению или простой структуре без таких определений пустой .cpp не создается. Шаблонный класс обычно определяется целиком в своем заголовке. Файл реализации одного класса не содержит определения методов другого самостоятельного класса; свободные функции группируются в отдельные файлы по назначению.

```
Report          -> include/report.h + src/report.cpp
StudentReport   -> include/student_report.h + src/student_report.cpp
GradeRecord     -> include/grade_record.h
StudentSummary  -> include/student_summary.h
ReportStatus    -> include/report_status.h
```

В полном примере выше Report и StudentReport показаны в двух отдельных рамках с именами файлов. Если для задачи понадобится ReportStatus, он будет объявлен в третьем файле report_status.h и подключен там, где используется. Перечисление не размещается рядом с Report только потому, что оно короткое.

6. Переменные, типы и значения

6.1. Объявление рядом с первым использованием

Переменная создается в минимальной необходимой области видимости и сразу получает начальное значение. Неиспользуемые переменные удаляются. Изменять одну переменную, по очереди придавая ей разные смыслы, не следует.

```
int validGradeCount = 0;
for (int grade : grades)
{
    if (grade >= c_minGrade && grade <= c_maxGrade)
    {
        ++validGradeCount;
    }
}
```

6.2. const и constexpr

Если значение дальше не меняется, применяется const. Если оно должно быть константным выражением и может быть вычислено при компиляции, применяется constexpr. Не нужно снимать const через const_cast, чтобы обойти дизайн интерфейса.

```
constexpr int c_maxGrade = 5;
const std::size_t studentCount = students.size();
```

6.3. Выбор формы инициализации

Для простых значений используется понятная запись `= значение`; `{}` удобно для пустого состояния, объектов и защиты от сужения. Для контейнеров учитывается выбор перегрузки: фигурные скобки могут создать список элементов, а круглые — нужное количество элементов. Синтаксис выбирается по смыслу.

```
std::vector<int> grades{4, 5, 3};
std::vector<int> emptyGrades(3, 0);
int studentCount{};
```

Первая строка создает три заданные оценки; вторая — три нуля. `std::vector<int>{3, 0}` создал бы два элемента, а не три.

6.4. auto и явный тип

auto применяется, когда тип понятен из правой части или его запись мешает чтению: `make_unique`, итераторы, сложные библиотечные типы. Явный тип полезен для единиц, вычислений и интерфейсов. auto сам по себе не сохраняет ссылку: для ссылки нужны `auto&` или `const auto&`.

```
auto report = std::make_unique<StudentReport>(grades);
const auto& reportGrades = report->GetGrades();
double totalGrade = 0.0;
```

6.5. Целые размеры и преобразования

Для размера стандартного контейнера используется `std::size_t` либо вывод типа. Отрицательное пользовательское число проверяется до преобразования в беззнаковый тип. Перед потенциально сужающим преобразованием проверяются границы. Применяются именованные преобразования C++, а не C-style cast. static_cast явно записывает намерение, но сам не проверяет переполнение.

```
double averageGrade = totalGrade / static_cast<double>(grades.size());
```

В примере отдельно проверено, что список не пуст. Целочисленное деление не используется там, где нужен дробный результат.

7. Функции, условия и циклы

7.1. Одна понятная задача и ранний выход

Функция выполняет одну связную задачу. Если для ее имени приходится перечислять несвязанные действия, ее следует разделить. Фиксированного лимита строк нет: вложенность и количество обязанностей важнее числа. Проверки недопустимого входа располагаются в начале; ранний return или исключение уменьшают вложенность.

```
if (grades.empty())
{
    throw std::domain_error("Average requires at least one grade");
}
```

7.2. Как передавать параметры

Недорогие значения, например int и bool, передаются по значению. Большой объект для чтения обычно передается по const-ссылке. Неконстантная ссылка означает изменение существующего объекта. Указатель может обозначать необязательный объект, для которого nullptr имеет отдельный смысл. Если объект сохраняет переданные данные, передача по значению с последующим move может быть удобнее набора перегрузок. Выбор зависит от назначения, а не только от размера типа.

```
void ValidateGrades(const GradeList& grades);
void SortGrades(GradeList& grades);
```

Не возвращается ссылка или указатель на локальную переменную. Время жизни заимствованного объекта должно покрывать все использования. Необязательный результат можно выразить `std::optional`; множество параметров одинакового типа лучше заменить понятной структурой, если их легко перепутать.

7.3. Логические условия без скрытых действий

Проверки bool пишутся непосредственно, без `== true`. Присваивание не прячется внутрь if. Усложненные условия получают именованные части; скобки добавляются для ясности сочетаний `&&` и `||`. Тернарный оператор подходит для короткого выбора значения, а не для нескольких побочных действий.

```
const bool hasValidGrade = grade >= c_minGrade && grade <= c_maxGrade;
if (hasValidGrade)
{
    AddGrade(grade);
}
```

7.4. Range-for, индекс и копирование

Если индекс не нужен, применяется range-for. Небольшие числа можно копировать; сложный объект для чтения берется по const-ссылке. Для изменения — по ссылке. При индексировании проверяются границы и не смешиваются неосознанно signed и unsigned. Размер контейнера нельзя уменьшать в цикле так, чтобы индекс или итератор стали недействительными.

```
for (const auto& student : students)
{
    PrintStudent(student);
}
```

7.5. switch и enum class

Ветви оформляются последовательно. Неявное проваливание запрещено: используется break, return, throw или явный `[[fallthrough]]`, если это намерение. Если внутри case нужна локальная переменная, создается отдельный блок. Для перечисления обрабатываются все допустимые значения; default не используется только ради подавления предупреждения о забытом новом значении.

```
switch (reportStatus)
{
case ReportStatus::Ready:
    PrintReport();
    break;
case ReportStatus::Empty:
    PrintEmptyMessage();
    break;
case ReportStatus::Failed:
    PrintError();
    break;
}
```

7.6. Лямбда и захват

Для короткой локальной операции подходит лямбда. Захваты перечисляются явно, если это помогает понять зависимость. Ссылка в захвате должна оставаться действительной до вызова; лямбда, переживающая локальную переменную, не захватывает ее по ссылке. Длинная логика получает отдельную именованную функцию.

```
constexpr int c_passingGrade = 3;
const auto isPassingGrade = [](int grade)
{
    return grade >= c_passingGrade;
};
```

Сам объект лямбды является переменной, поэтому `isPassingGrade` имеет camelCase, хотя вызов внешне похож на вызов функции.

8. Ресурсы, ошибки и корректность

8.1. Ресурсы принадлежат объектам

Для строк используется std::string, для динамических последовательностей — стандартные контейнеры. Файл, блокировка и память освобождаются владельцем автоматически: принцип RAII. Для единственного владельца полиморфного объекта используется unique_ptr и make_unique. shared_ptr применяется только при реальной необходимости совместного владения. Обычный указатель или ссылка обычно не владеет объектом.

```
auto report = std::make_unique<StudentReport>(GradeList{4, 5, 3});
```

Ручные new/delete, malloc/free и массивы переменной длины не используются в обычном прикладном коде курса. Они допустимы только если непосредственно изучаются в задании. nullptr применяется вместо NULL и числового нуля для отсутствующего указателя.

8.2. Проверка ввода и обработка ошибки

Пользовательский ввод проверяется: удалось ли чтение, допустимы ли диапазон, размер и формат. Ошибка ввода не замалчивается и не превращается произвольно в корректное значение. Ожидаемое отсутствие результата может выражаться через optional или согласованный тип результата. Исключение подходит, если операцию невозможно завершить и ошибка передается вызывающему коду. В одном интерфейсе применяется последовательная модель ошибок.

```
int studentCount = 0;
if (!(std::cin >> studentCount) || studentCount < 0)
{
    throw std::invalid_argument("Student count must be non-negative");
}
```

Стандартное исключение перехватывается по `const std::exception&`. Пустой catch запрещен. assert проверяет предположение разработчика и может исчезнуть в release-сборке: он не заменяет проверку пользовательского ввода.

8.3. Строки и вещественные числа

std::string сравнивается по содержимому. Сравнение двух `const char*` через == сравнивает адреса, а не текст. Для вещественных вычислений способ сравнения определяется задачей: допуск учитывает масштаб и требования; не выбирается универсальная константа на все случаи. Для ввода, где недопустимы NaN и бесконечность, проверяется `std::isfinite`.

```
const std::string expectedName = "Alex";
if (studentName == expectedName)
{
    PrintStudent();
}
```

8.4. Ограничения состояния и инкапсуляция

Класс поддерживает свои ограничения после завершения каждой открытой операции. Если оценки могут быть только от 2 до 5, конструктор и методы изменения проверяют это. Открытая изменяемая ссылка на внутренний список позволила бы обойти проверку, поэтому наружу выдается только чтение. Наследование применяется для подстановки одного поведения вместо другого; простое повторное использование реализации часто удобнее выразить композицией.

```
[[nodiscard]] const GradeList& GetGrades() const noexcept;
```

8.5. Переносимость и зависимости

Используются стандартные заголовки, а не `<bits/stdc++.h>`. Типы из std обозначаются явно. Не предполагается, что int всегда имеет конкретный размер; для двоичного формата при необходимости используются типы `<cstdint>` с подходящими проверками. Зависимость от ОС изолируется в соответствующем модуле. Глобальное изменяемое состояние, скрытые побочные действия макросов и необоснованная оптимизация не применяются.

9. Что проверяется автоматически и что читает преподаватель

clang-format проверяет отступы, пробелы, скобки, переносы и часть порядка include. clang-tidy проверяет настроенные категории имен, отсутствие некоторых типичных ошибок, override, nullptr, скобки у условий и часть правил инициализации. Он не определяет, удачно ли выбрано английское слово, нужен ли классу сеттер и удобен ли его интерфейс. Формат имен файлов, отсутствие комментариев и установленный порядок членов класса проверяются отдельно при чтении работы. Эти настройки не являются доказательством корректности алгоритма.

Перед сдачей:

1. Код соответствует таблице; у всех собственных методов PascalCase.
2. Каждый самостоятельный тип находится в отдельном заголовке; класс читается от public к private, поля находятся в конце private.
3. Наследование, virtual, override, final и доступ выбраны по смыслу.
4. Нет комментариев, закомментированного кода и неиспользуемых фрагментов.
5. Форматирование и выбранные проверки clang-tidy проходят без ошибок.
6. Программа собирается в выбранном стандарте без предупреждений проекта.
7. Проверены обычный случай, пустые данные, границы и неверный ввод.
8. Отчет или README содержит необходимые внешние пояснения решения.

10. Источники и область применения

Нормативные правила курса изложены в этом документе. Источники подтверждают выбранную основу и поведение инструментов; дополнительные рекомендации языка сформулированы для учебных работ и не означают буквального принятия всех правил перечисленных проектов.

- Microsoft DirectXTK — Naming conventions: основа именования типов, функций, методов и переменных.
- LLVM — ClangFormatStyleOptions: Microsoft, Allman и параметры оформления.
- LLVM — Identifier naming: категории автоматической проверки имен.
- LLVM — Use override: проверка переопределений.
- LLVM — Virtual class destructor: модели доступа и виртуальности деструктора.
- C++ Core Guidelines: рекомендации по интерфейсам, ресурсам и времени жизни.

Версия соглашения: 12 октября 2026 года. Готовые .clang-format и .clang-tidy являются частью комплекта. Дополнительные требования отдельного задания имеют приоритет в пределах этого задания.